CY Tech Stages - Rapport de Livrable

Plateforme de Gestion des Stages ING1

Génie Informatique - Groupe 2 - Groupe 11

- **Titouan Ancelin :** Chef de projet & Développement backend
- **Hedy Ouerghi :** Développement frontend & intégration
- **Jérémy Garra :** Base de données, tests & documentation

Sommaire

- **1. Introduction et contexte du projet**
- **2. Présentation de l'équipe et organisation**
- **3. Architecture technique**
- **4. Modèle de données (17 tables)**
- **5. Fonctionnalités - Étudiant & Entreprise**
- **6. Fonctionnalités - Tuteur, Jury & Administrateur**
- **7. Points techniques**
- **8. Bilan, tests et qualité du code**

1. Introduction et contexte du projet

Plateform Stages CY Tech est une plateforme web de gestion des stages développée dans le cadre du module "Projet Développement Web" de la filière ING1 à CY Tech. L'objectif du projet est de numériser et centraliser l'ensemble du processus de gestion des stages de l'école depuis la publication d'offres par les entreprises jusqu'à la soutenance devant jury, en passant par le suivi pédagogique assuré par les tuteurs pédagogiques.

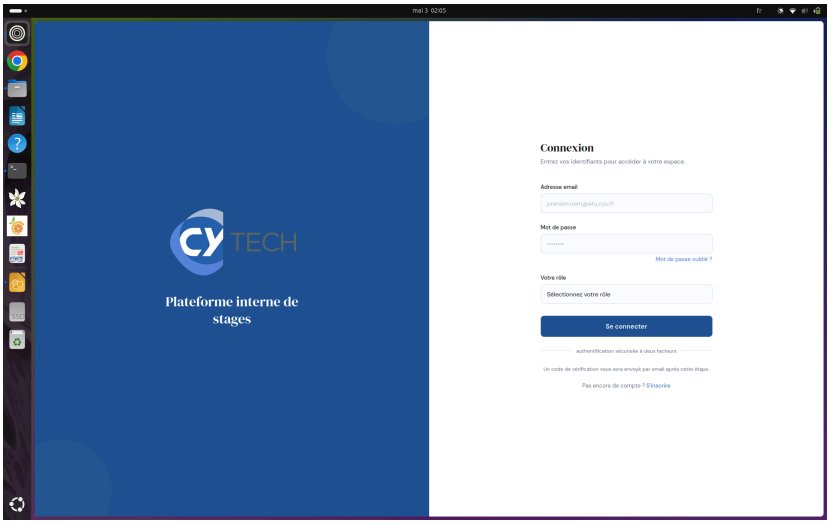
Contexte et problématique

Avant ce projet, la gestion des stages reposait sur des échanges par email et des documents papier, générant des pertes d'information, des délais et un manque de visibilité pour toutes les parties. La Plateforme Stage CY Tech répond à ce besoin en proposant une interface unifiée pour cinq types d'acteurs : les étudiants, les entreprises, les tuteurs pédagogiques, les membres du jury et l'administrateur de l'école.

Périmètre fonctionnel

La plateforme couvre l'intégralité du cycle de vie d'un stage ING1 :

- Inscription et authentification avec double facteur (2FA par email).
- Publication et recherche d'offres de stage (filtres : durée, lieu, filière, mots-clés).
- Dépôt de candidatures avec CV et lettre de motivation (PDF/DOCX, max 5 Mo).
- Workflow de convention : étudiant → entreprise + tuteur (en parallèle) → admin.
- Suivi pédagogique : cahier de stage, missions, commentaires bidirectionnels.
- Notation par le tuteur et par le jury après affectation par l'administrateur.
- Gestion documentaire : dépôt rapport, résumé, fiche d'évaluation.
- Tableau de bord admin : validation de comptes, archivage, affectations.
- Profil public de chaque utilisateur connecté (/utilisateur/{id}).
- Journalisation automatique de toutes les actions utilisateurs (USER_ACTIVITE).



2. Présentation de l'équipe et organisation

Composition de l'équipe

- **Titouan Ancelin (Chef de projet / Backend) :** Coordination générale et planification Agile (sprints Milanote). Architecture Laravel (controllers, models, middleware, routes). Logique métier (conventions, encadrement, affectations jury). Déploiement local et gestion de la base de données MySQL.
- **Hedy Ouerghi (Frontend / Intégration) :** Conception et intégration des vues PHP. Mise en place du design system (CSS custom, grille KPI, cards). Formulaires réactifs, validation côté client, messages flash. Interfaces profil étudiant, documents, cahier de stage.
- **Jérémy Garra (Base de données / Tests / Documentation) :** Conception du MCD/MLD et écriture du schéma SQL initial. Migrations Laravel et seeders de données de test. Tests fonctionnels manuels de tous les workflows. Rédaction des livrables et documentation technique.

Méthodologie et outils

L'équipe a adopté une méthode Agile inspirée de Scrum, avec des sprints de deux semaines. Le backlog et la roadmap ont été gérés sur Milanote. Le versionning a été assuré via Git avec GitHub comme dépôt distant. L'environnement de développement est entièrement local (PHP 8.3, MySQL 8, serveur artisan). L'éditeur utilisé par tous les membres est Visual Studio Code avec l'extension PHP Intelephense, indispensable pour l'auto-complétion et la détection d'erreurs en temps réel."

Domaine	Outil / Technologie
Gestion de projet	Milanote
Versionning	Git + GitHub (branches, pull requests)
IDE	Visual Studio Code + PHP Intelephense
Hébergement	Serveur de développement local (php artisan serve)
SGBD	MySQL 8 - base : stages_cytech
Framework	Laravel 11 - PHP 8.3 - Bootstrap 5 (CDN)
Emails	Driver "log" Laravel (storage/logs/laravel.log)

Étapes de réalisation

Le projet s'est déroulé sur 8 semaines, structuré en 5 phases clés :

- **Phase 1 : Cadrage (S1-S2) :** Analyse approfondie du cahier des charges, définition des fonctionnalités et choix de la stack technique (Laravel 11 / PHP 8.3).
- **Phase 2 : Conception (S2-S3) :** Création des maquettes (wireframes), modélisation de la base de données (MCD/MLD) et mise en place de la structure Git.
- **Phase 3 : Développement Backend (S4-S6) :** Création de la base MySQL, développement du système d'authentification et de la logique métier (offres, stages, dépôt de documents).
- **Phase 4 : Front-End (S4-S6) :** Intégration HTML/CSS avec Bootstrap, validation des formulaires en JS et optimisation responsive (Mobile-First).
- **Phase 5 : Tests & Finalisation (S7-S8) :** Tests fonctionnels, vérification de la compatibilité navigateurs, rédaction du rapport final et préparation de la soutenance.

3. Architecture technique

Stack technologique

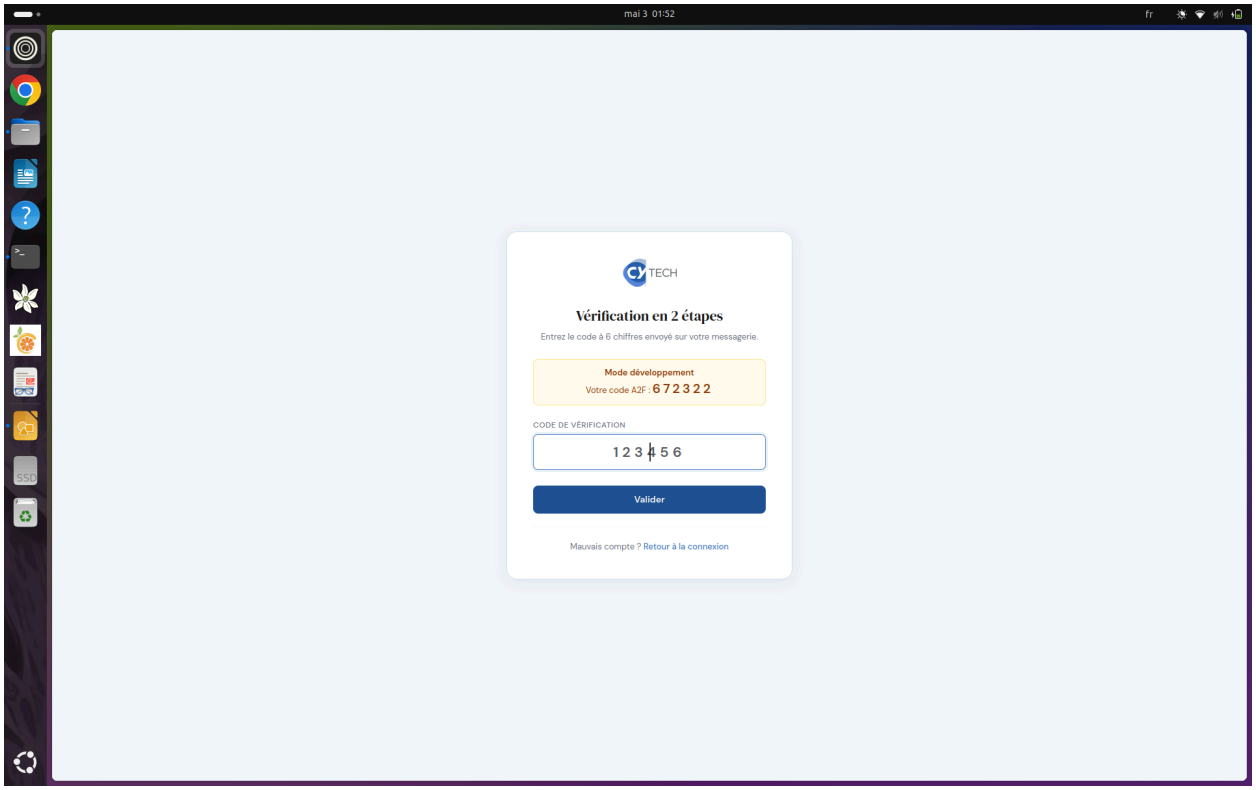
La plateforme repose sur Laravel 11 (PHP 8.3) selon le pattern MVC. Les vues sont directement développées en PHP pur avec un système de layout basé sur ob_start() / ob_get_clean(), (qui permet de mettre le code HTML en mémoire tampon avant de l'injecter dans la page principale), afin de se conformer aux contraintes pédagogiques. La session est gérée en mode fichier (SESSION_DRIVER=file) pour éviter la dépendance à une table 'sessions' en base de données.

Répertoire	Contenu
app/Http/Controllers/	7 controllers : Auth, Dashboard, Etudiant, Entreprise, Tuteur, Jury, Admin
app/Http/Middleware/	CheckAuth (auth custom session), LogActivite (journalisation auto)
app/Models/	15 modèles Eloquent avec noms de tables UPPERCASE
resources/views/	25+ vues PHP pures organisées par rôle (+ layouts/app.php)
database/migrations/	21 migrations en ordre chronologique strict
database/seeders/	Database Seeder : 6 utilisateurs + 2 offres de stage initiales
routes/web.php	40+ routes groupées par middleware auth.check
storage/app/uploads/	Fichiers déposés (CV, lettres, rapports, fiches)
storage/app/conventions/	Fichiers convention signée (flux séparé)

Authentification et sécurité

L'authentification est entièrement personnalisée (pas de packages tiers). Le flux est le suivant : saisie email/mot_de_passe → vérification bcrypt → envoi code 2FA par email → saisie du code sur /a2f → suppression du code → session établie. Les routes protégées sont filtrées par un middleware (CheckAuth), qui agit comme un sas de sécurité pour vérifier la session avant de laisser passer la requête au contrôleur."

- Mots de passe hashés avec bcrypt (Hash::make).
- Codes 2FA à usage unique stockés dans AUTH_CODE (expirés après vérification).
- Validation CSRF sur tous les formulaires POST.
- Accès aux routes vérifié par middleware + vérifications contextuelles dans les controllers.
- Upload : validation MIME type + taille max côté serveur.



4. Modèle de données (17 tables)

La base de données MySQL comprend 17 tables (noms en MAJUSCULES pour respecter la convention du schéma SQL initial). Toutes les tables ont été créées via des migrations Laravel versionnées,(ce qui permet de versionner le schéma de la base comme on le fait avec Git pour le code), garantissant la reproductibilité de l'environnement. Le jeu de données initial est généré par le Database Seeder, un script qui remplit automatiquement la base pour les tests.

Table	Description
UTILISATEUR	Tous les comptes (nom, prénom, email, mot_de_passe bcrypt, rôle, validé)
AUTH_CODE	Codes 2FA temporaires liés à un utilisateur
ETUDIANT	Profil étudiant (numéro, filière, promotion, id_utilisateur)
TUTEUR	Profil tuteur (département, id_utilisateur)
JURY	Profil jury (spécialité, id_utilisateur)
ENTREPRISE	Profil entreprise (nom, secteur, adresse, id_utilisateur)
OFFRE_STAGE	Offres publiées (titre, description, compétences, durée, lieu)

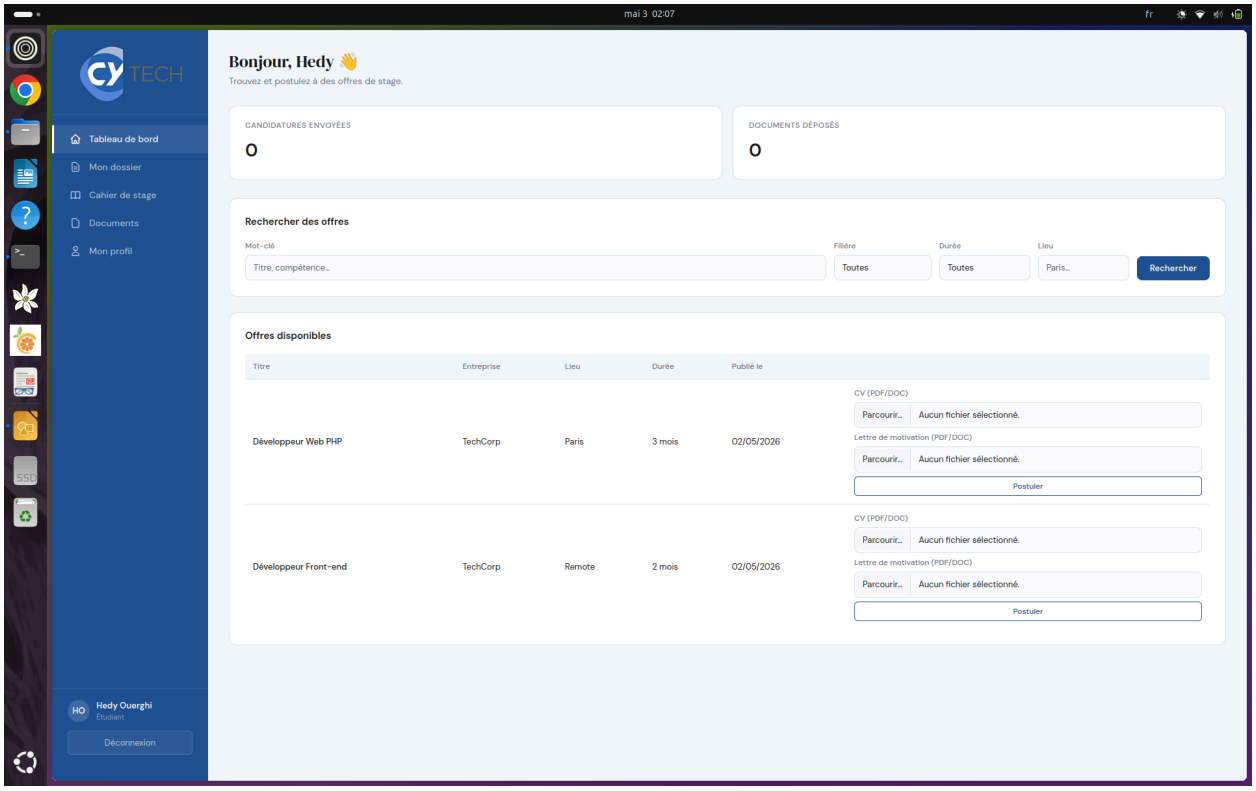
Table	Description
CANDIDATURE	Candidatures (statut : en_attente/validée/refusée/archivée)
CONVENTION	Conventions (chemin fichier, statuts étudiant/entreprise/tuteur/admin)
DOCUMENT	Documents déposés (type, chemin, nom original, id_candidature)
SUIVI	Notes attribuées par tuteur et jury (note /20, commentaire)
COMMENTAIRE	Messages bidirectionnels sur une candidature
DEMANDE_ENCADREMENT	Demandes d'affectation tuteur soumises par les tuteurs
AFFECTATION_JURY	Affectations jury candidature par l'admin
CAHIER_STAGE	Entrées quotidiennes du journal de stage de l'étudiant
MISSION	Missions attribuées par l'entreprise à l'étudiant
USER_ACTIVITE	Journal : chaque accès (GET) et action (POST+) horodatés

5. Fonctionnalités - Étudiant & Entreprise

5.1 Espace Étudiant

L'étudiant dispose d'un tableau de bord centralisé avec indicateurs clés (nombre de candidatures, de documents), un moteur de recherche d'offres et des accès directs à toutes ses sections.

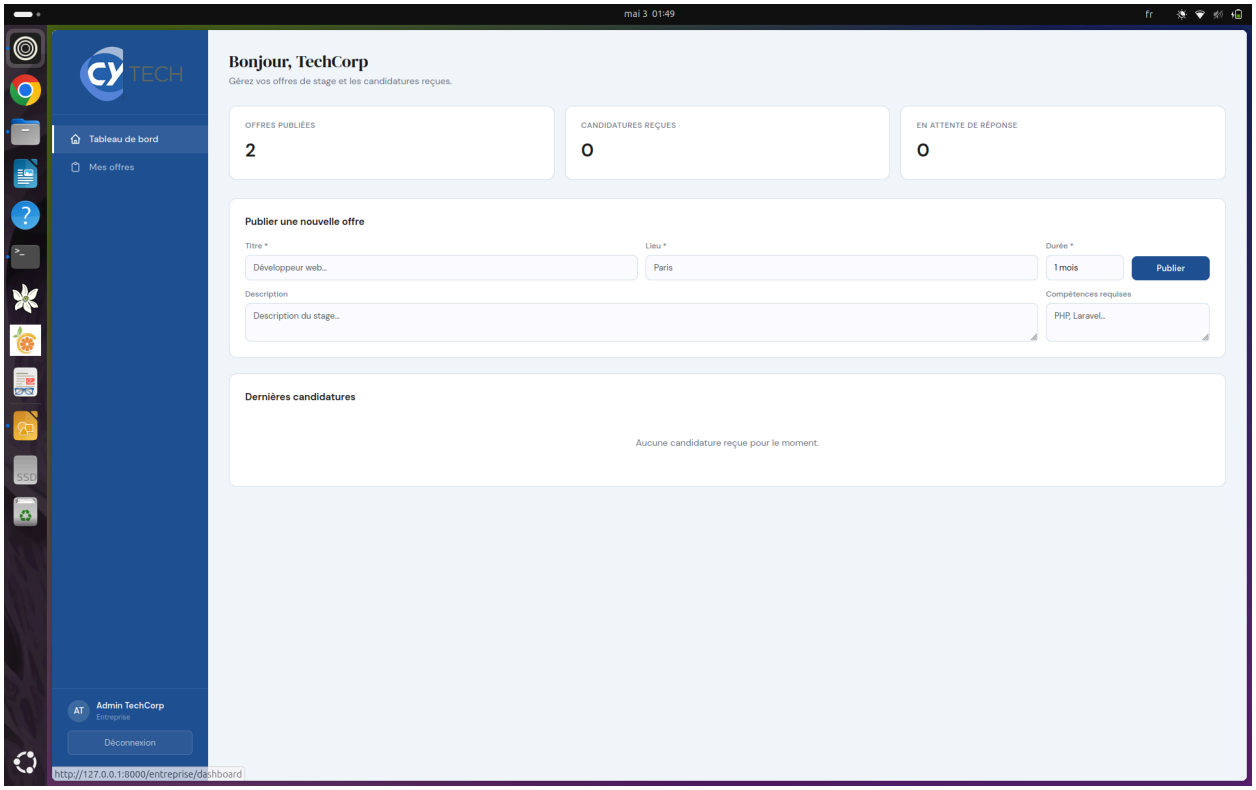
- Recherche d'offres avec 4 filtres cumulés (mot-clé, durée, lieu, filière).
- Postuler : upload simultané CV + lettre de motivation (PDF/DOCX, max 5 Mo).
- Mon dossier : historique des candidatures, statuts, commentaires, conventions.
- Documents : dépôt rapport de stage, résumé, fiche d'évaluation après validation.
- Convention : upload du fichier signé en attente entreprise + tuteur → admin.
- Cahier de stage : ajout d'entrées journalières (date + contenu libre).
- Mon profil : modification email, filière, promotion, mot de passe (nom/prénom verrouillés).
- Annulation candidature (uniquement si statut 'en_attente').
- Téléchargement de ses propres documents déposés (vérification de propriété).
- Soumission d'une requête à l'administrateur pour l'ajout d'une nouvelle filière/formation si celle-ci est absente de la liste lors d'une recherche.



5.2 Espace Entreprise

L'entreprise gère ses offres de stage et suit les candidatures reçues. Elle participe au workflow de convention et peut attribuer des missions aux étudiants dont la candidature a été validée.

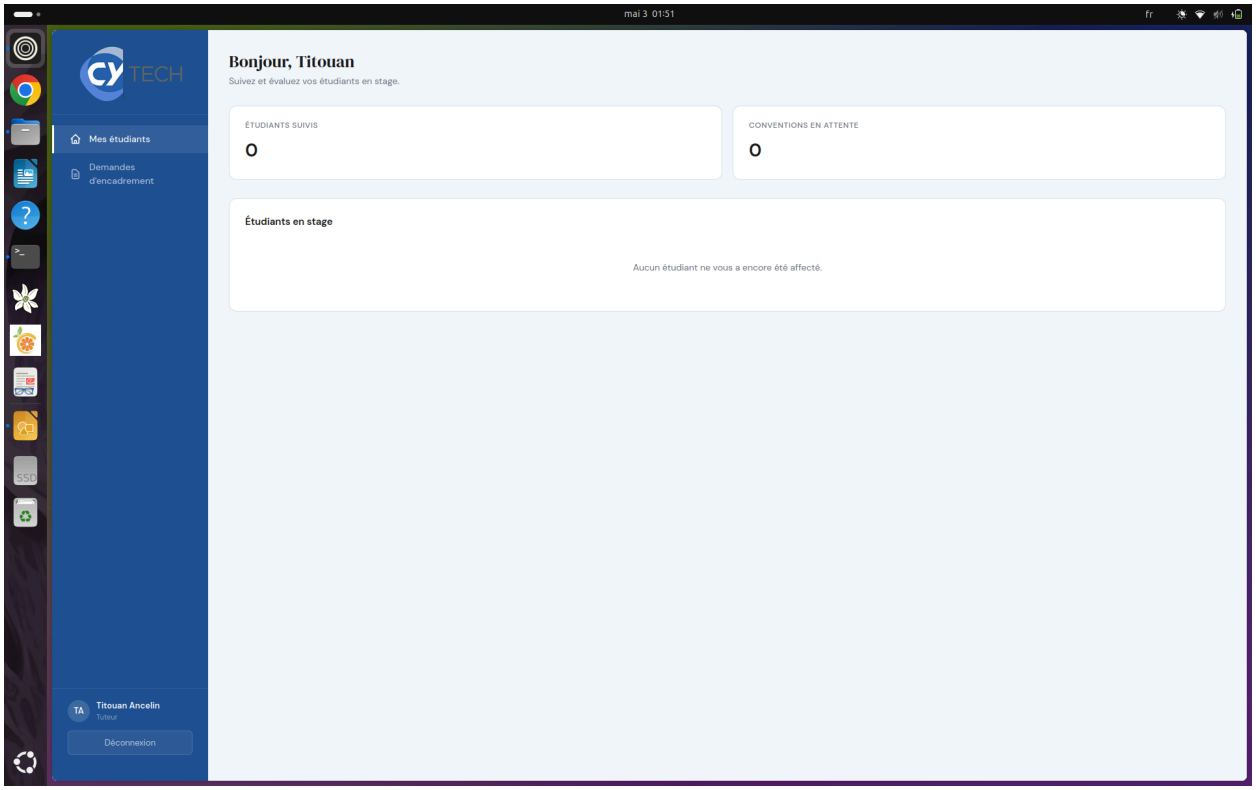
- Publication d'offres : titre, description, compétences, durée, lieu.
- Suppression d'une offre (avec suppression en cascade des candidatures).
- Tableau des candidatures reçues par offre avec détail CV/lettre.
- Validation / Refus de candidature avec notification (log mail).
- Commentaires sur une candidature (bidirectionnel étudiant ↔ entreprise).
- Validation de convention : signature numérique → workflow admin.
- Création de missions pour un étudiant (titre, description, date fin).
- Clôture de mission avec statut terminé.
- Téléchargement des documents soumis par les étudiants.



6. Fonctionnalités - Tuteur, Jury & Administrateur

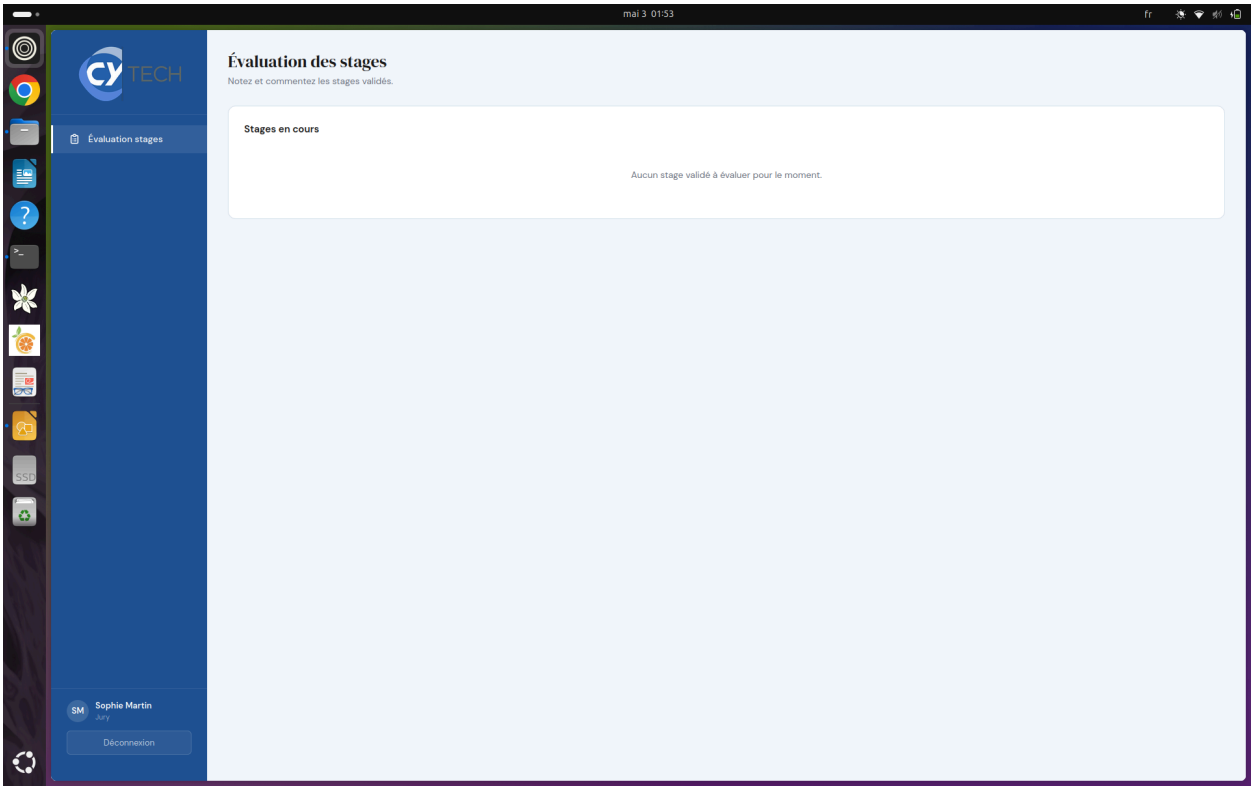
6.1 Espace Tuteur

- Tableau de bord : liste des étudiants encadrés (candidatures affectées).
- Demande d'encadrement : le tuteur soumet une demande pour encadrer un étudiant.
- Validation de convention : co-validation avec l'entreprise (workflow parallèle).
- Commentaires sur les dossiers des étudiants encadrés.
- Attribution d'une note /20 à l'étudiant (stockée dans SUIVI).
- Accès au cahier de stage d'un étudiant encadré (lecture seule).
- Téléchargement de la convention signée.



6.2 Espace Jury

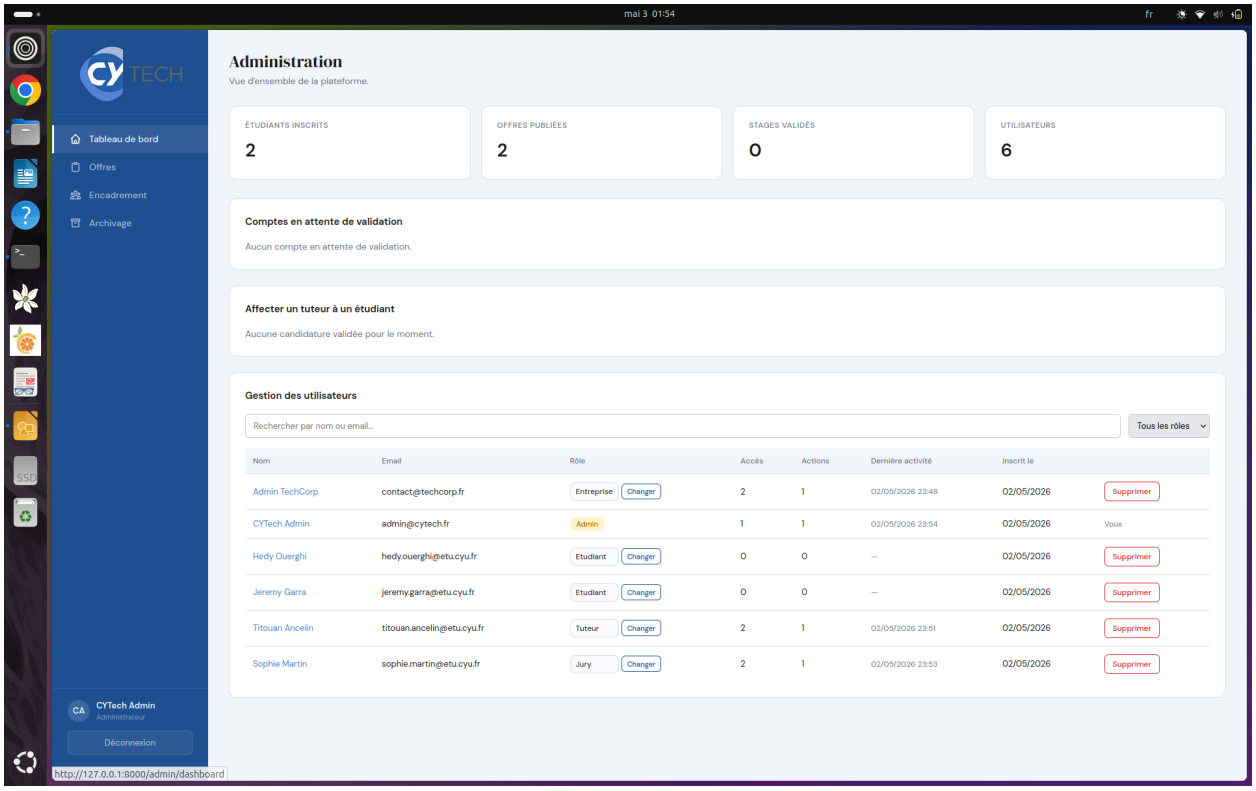
- Tableau de bord : liste des candidatures affectées par l'admin.
- Attribution d'une note /20 (distincte de la note tuteur).
- Commentaires sur le dossier de l'étudiant évalué.



6.3 Espace Administrateur

L'administrateur dispose du niveau de contrôle le plus élevé sur la plateforme. Son tableau de bord consolide toutes les informations critiques : comptes en attente, conventions à valider, demandes d'encadrement, affectations jury.

- Validation / refus de comptes (entreprises, tuteurs, jurys soumis à validation).
- Modification du rôle d'un utilisateur (ex: étudiant → tuteur).
- Suppression d'un utilisateur avec cascade complète.
- Modération des offres de stage publiées.
- Archivage d'une candidature validée (conservation des données).
- Désarchivage : réactivation d'un étudiant archivé (réutilisation des infos).
- Affectation d'un tuteur à un étudiant (via demandes d'encadrement).
- Acceptation / refus des demandes d'encadrement soumises par les tuteurs.
- Affectation d'un jury à une candidature + retrait d'affectation.
- Validation finale de convention (après signatures entreprise + tuteur).
- Gestion et validation des requêtes étudiantes pour l'ajout de nouvelles filières/formations.



7. Points techniques

7.1 Développement natif des vues en PHP

L'ensemble des vues est codé nativement en PHP. Le système de layout utilise ob_start() / ob_get_clean() pour capturer le contenu de chaque vue dans des variables $nav et $content, injectées dans app.php. La fonction `e()` est utilisée pour l'échappement XSS (elle neutralise tout code HTML ou JavaScript malveillant qu'un utilisateur essaierait d'injecter via un formulaire). Le projet démontre une conception maîtrisée sans recours à un moteur de template additionnel.

7.2 Workflow de convention à trois validateurs

La convention suit un workflow en deux phases : (1) l'étudiant uploade le fichier signé (statut_etudiant = 'signe') ; (2) l'entreprise ET le tuteur valident en parallèle (ordre indépendant) ; (3) l'admin valide en dernier (statut_admin = 'valide'). Ce workflow est géré par un seul enregistrement CONVENTION mis à jour à chaque étape.

7.3 Authentification 2FA personnalisée

Après la vérification du mot de passe, un code numérique à 6 chiffres est généré (random_int), stocké dans AUTH_CODE avec horodatage, et envoyé par email. L'utilisateur saisit le code sur la route sécurisée. Après vérification, le code est supprimé et la session est établie.

7.4 Journalisation automatique (LogActivite)

Le middleware LogActivite s'exécute sur chaque requête authentifiée. Il enregistre dans USER_ACTIVITE : l'id utilisateur, le type (accès pour GET, action pour POST/PUT/PATCH/DELETE), l'URL, et un horodatage automatique. Cette journalisation est transparente et garantit la traçabilité.

7.5 Gestion sécurisée des fichiers

Tous les fichiers uploadés transitent par Storage::store() (disque 'local'). Les noms originaux sont préservés dans la base de données. Le téléchargement se fait via Storage::download() avec vérification stricte de propriété ; aucun accès direct par URL n'est possible.

8. Bilan, tests et qualité du code

8.1 Couverture fonctionnelle

L'ensemble des fonctionnalités décrites dans le cahier des charges a été implémenté. Le tableau ci-dessous récapitule les groupes de fonctionnalités et leur statut.

Groupe	Fonctionnalité	Statut
A	Upload documents restitution (rapport, résumé, fiche évaluation)	**Complet**
A	Filtre filière sur les offres de stage	**Complet**
A	Modification profil étudiant (email, filière, promotion, MDP)	**Complet**
A	Convention parallèle (entreprise + tuteur) puis validation admin	**Complet**
B	Commentaires bidirectionnels (étudiant, entreprise, tuteur, jury)	**Complet**
C	Cahier de stage (entrées journalières, visible tuteur)	**Complet**
C	Missions attribuées par l'entreprise aux étudiants	**Complet**
D	Modification de rôle par l'admin	**Complet**
D	Archivage / désarchivage avec conservation des données	**Complet**
D	Page profil public (/utilisateur/{id})	**Complet**
E	Envoi email (driver log Laravel) code 2FA + notifications	**Complet**
E	Journalisation accès + actions (middleware LogActivite)	**Complet**

8.2 Qualité du code et conventions

- Code épuré et commenté de manière professionnelle.
- Séparation des responsabilités : logique métier isolée dans les controllers.
- Validation systématique côté serveur sur tous les formulaires ($request->validate()).
- Protection CSRF sur tous les formulaires POST (token Laravel).

8.3 Accessibilité, UX et Optimisation

Afin de garantir une expérience utilisateur (UX) optimale et conforme aux exigences, plusieurs mesures ont été mises en place :

- **Conception Responsive :** L'interface a été pensée *Mobile-First* puis adaptée aux écrans larges avec des grilles flexibles.
- **Accessibilité :** Respect des normes WCAG (Web Content Accessibility Guidelines) avec l'utilisation systématique de balises sémantiques, l'ajout de textes alternatifs sur toutes les images, et le respect de contrastes de couleurs adéquats.

- **Performances et Optimisation :** Les médias ont été compressés et le chargement différé (lazy loading) a été activé pour éviter les ralentissements et mise en place d'un système de cache (headers et serveur) pour les fichiers statiques et données répétitives
- **Tests de compatibilité :** La plateforme a été testée avec succès sur les principaux navigateurs (Chrome, Firefox, Safari) et son rendu a été validé sur plusieurs tailles d'écrans (mobile, tablette, desktop).

8.4 Conclusion et perspectives

Le projet Plateforme Stages CY Tech constitue une application web complète et fonctionnelle, couvrant l'intégralité du cahier des charges. L'équipe a su relever le défi technique de développer une architecture robuste en PHP pur sur une base Laravel, tout en maintenant un code propre et une interface ergonomique. Les différentes étapes du projet, de la phase de planification jusqu'à la réalisation finale, nous ont permis d'aboutir à un résultat fonctionnel répondant aux besoins de l'école.

Les perspectives d'évolution incluent : un système de notifications en temps réel via websockets (pour recevoir des alertes sans avoir à rafraîchir la page), une API REST pour une éventuelle application mobile, un déploiement sur serveur de production (Docker + Nginx), et l'ajout de tests unitaires et fonctionnels automatisés. Le projet constitue une base solide pour une plateforme de gestion des stages à l'échelle de l'établissement.